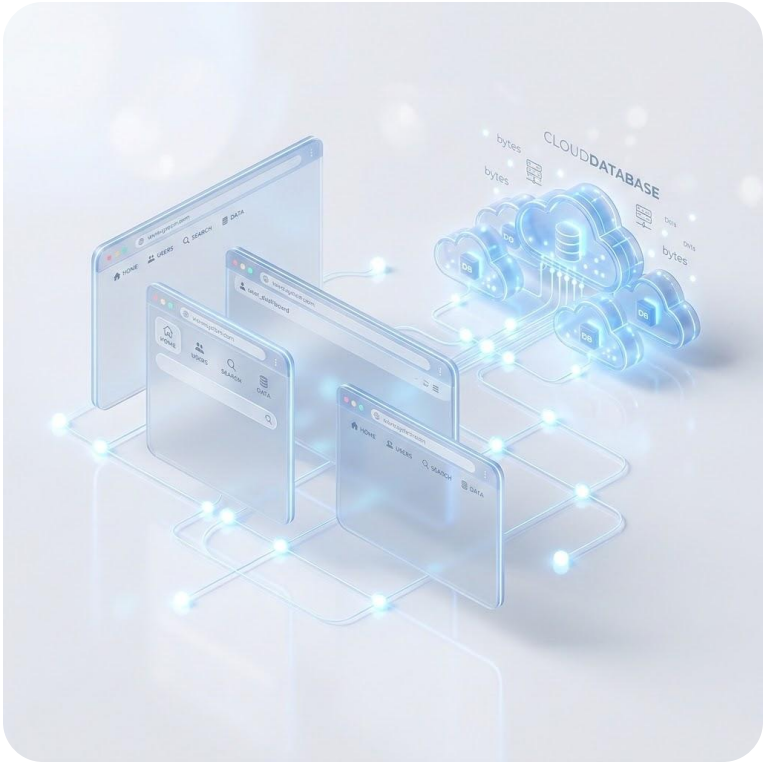
Departamento de Informática
UFV Universidade Federal de Viçosa

INF 321

Projeto e Desenvolvimento de Sistemas para a Web

Unidade 2.3:
Desenvolvimento Front-End |> JavaScript

Prof. Lucas Vegi



JavaScript

Programação para Front-end

Até agora:

HTML

Função
estrutura

CSS

Função
apresentação

O próximo passo:

JavaScript

Função
comportamento

Uma página sem JavaScript

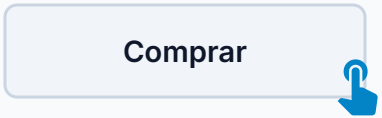
O comportamento padrão do HTML

1. Declaração do Elemento

Código HTML do Botão

```
<!-- Botão simples -->
<button>Comprar</button>
```

Como o navegador exibe:



2. Interação sem Script

O que acontece ao clicar?

Absolutamente Nada!

Sem códigos JavaScript associados para capturar e tratar o evento de clique, o botão é apenas um elemento estático e inerte na interface.

Com JavaScript

Programando comportamentos e ações na página

1. O FLUXO DE EXECUÇÃO

Como a ação é programada:

Clique no botão
↓
JavaScript executa o código
↓
Página é modificada dinamicamente

2. EXEMPLO PRÁTICO

Simulação de Interatividade:

Comprar
clique

Produto adicionado ao carrinho!

Conceito Chave: O JavaScript permite tornar as páginas web verdadeiramente interativas, respondendo a ações do usuário em tempo real.

O que é JavaScript?

Linguagem de programação para comportamento e interatividade

Conceito Principal

É uma linguagem de programação amplamente utilizada para adicionar **comportamento dinâmico** e **interatividade** às páginas Web.

Aplicações Práticas

No navegador, podemos usar para:

- Responder a eventos do usuário
- Modificar elementos HTML e CSS
- Validar formulários
- Realizar cálculos
- Manipular e estruturar dados
- Fazer requisições HTTP
- Atualizar a interface dinamicamente

Onde o JS é executado?

Ambientes de execução do código JavaScript

No Navegador (Client-Side)

O ambiente principal do front-end. O navegador possui um **motor JavaScript** integrado que executa o código diretamente no cliente.

Papéis no desenvolvimento:

HTML: estrutura da página
CSS: apresentação visual
JS: comportamento dinâmico

No Servidor (Server-Side)

O JavaScript também é extremamente potente no back-end. Com o **Node.js**, o código roda diretamente no servidor do sistema.

Node.js e Ecossistema:

Execução fora do ambiente de browsers
Criação de APIs rápidas e escaláveis
Acesso direto ao sistema de arquivos

Adicionando JavaScript ao HTML

Como integrar comportamento de programação nas suas páginas web

MÉTODO INTERNO

Inserção Direta via Tag

Perfeito para scripts rápidos ou testes simples de comportamento diretamente no próprio arquivo HTML.

```
<script>
   console.log("Olá!");
</script>
```

Limitação: Dificulta a manutenção e reutilização do código em múltiplos arquivos HTML do projeto.

MÉTODO RECOMENDADO

Arquivo Externo Separado

Separação clara de responsabilidades estruturando seu projeto de forma limpa e organizada:

📁 Arquivos: `index.html` | `style.css` | `script.js`

```
<!-- Conexão no HTML -->
<script src="script.js"></script>
```

✓ **Vantagem: Código reutilizável, carregamento otimizado (cache) e arquivos organizados.**

JavaScript Externo

Onde posicionar a tag `<script>` para otimizar a performance

POSIÇÃO RECOMENDADA

No Final do Body (`</body>`)

Garante que todo o HTML seja renderizado antes de processar o JavaScript, evitando que a página pareça travada no carregamento inicial.

```
<body>
  <!-- Conteúdo da página -->
  <script src="script.js"></script>
</body>
```

OUTRA ALTERNATIVA

Dentro do Head (`<head>`)

Útil para scripts de rastreamento ou configurações críticas que precisam carregar antes da exibição da página.

⚠ Impacto na Performance:

Pode tornar o carregamento mais lento, pois o navegador interrompe a leitura do HTML para baixar e executar o script.

JavaScript Externo

Como organizar fisicamente e vincular os arquivos do seu projeto web

CÓDIGO FONTE

Conexão e Implementação

No HTML (vínculo externo):

```
<!-- index.html -->
<script src="script.js"></script>
```

No JavaScript (comportamento):

```
// script.js
console.log("Olá, mundo!");
```

ESTRUTURA DE DIRETÓRIO

Organização Física

Os arquivos devem residir na mesma pasta (ou caminhos relativos mapeados):

- meu-projeto/
  - index.html
  - style.css
  - **script.js** JS Externo

Assim como no CSS, separar o código facilita a organização e a manutenção a longo prazo.

Primeiros passos com JS: `console.log`

O console como ferramenta essencial de desenvolvimento

EXIBINDO INFORMAÇÕES

O Comando console.log

Podemos exibir e testar qualquer informação diretamente no painel de controle do desenvolvedor.

```
// Executando o comando
console.log("Olá, mundo!");
```

O console será uma ferramenta extremamente importante durante toda a sua jornada de desenvolvimento.

PASSO A PASSO

No Navegador

Para abrir e visualizar a saída do comando em tempo real nas ferramentas do desenvolvedor:

1. **Abra o DevTools (F12 ou Ctrl+Shift+I)**
2. **Navegue até a aba Console**

SAÍDA ESPERADA NO CONSOLE:

```
> Olá, mundo!
```



Variáveis em JavaScript

Como armazenar, utilizar e entender o comportamento de valores na memória

CONCEITO BÁSICO

Armazenando Valores

O JavaScript permite guardar dados na memória utilizando a palavra-chave `let` e imprimi-los com `console.log`.

```
let nome = "Lucas";
let idade = 30;

// Exibindo no console
console.log(nome);
console.log(idade);
```

Dica: Use nomes descritivos para facilitar a leitura.



COMPORTAMENTO DO JS

Tipagem Dinâmica

Não é necessário definir os tipos de dados explicitamente. O interpretador do JS descobre o tipo em tempo de execução automaticamente.

⚡ Estática com TypeScript

Para projetos grandes que demandam maior segurança e autocompletação, o **TypeScript** é utilizado como superset estático.

✓ **Flexibilidade JS:** Altere o tipo de valor de uma variável a qualquer momento.

A palavra-chave `const`

Quando utilizar e como se diferencia do `let` de forma prática

CONCEITO BÁSICO

Valores Imutáveis

Utilize const para declarar variáveis cujo valor não será reatribuído após a definição inicial.

```
const preco = 5000;
const produto = "Notebook";
```

✓ **Segurança:** Evita reatribuições acidentais ao longo do código.

DE FORMA PRÁTICA

Diferença de Comportamento

Entenda a dinâmica de alteração de valores entre as duas palavras-chave:

`let` → O valor pode ser alterado

Permite reatribuição a qualquer momento do fluxo.

`const` → Não será reatribuído

Gera um erro se você tentar atribuir um novo valor.

Tipos de Dados

Principais classificações, tipos primitivos comuns e estruturas em JavaScript

TIPOS PRIMITIVOS

Valores Comuns

Variáveis simples que armazenam um único valor diretamente na memória física.

```
let nome = "Notebook"; // string
let preco = 5000; // number
let disponivel = true; // boolean
```

⚡ **Primitivos:** São imutáveis e comparados diretamente por seu valor.

ESTRUTURAS & ESPECIAIS



Ausência e Coleções

Representação de ausência intencional de valor e agrupamento indexado de dados.

```
let produto = null; // ausência de valor
// Estrutura de dados (Array/Lista)
let produtos = ["Notebook", "Celular"];

console.log(produtos[0]); //"Notebook"
```

✓ **Objetos:** Arrays e nulos possuem comportamento de objeto em JavaScript.

Operações em JavaScript

Como realizar cálculos matemáticos e manipular valores utilizando variáveis

CONCEITO BÁSICO

Realizando Cálculos

O JavaScript permite processar operações aritméticas diretamente com variáveis.

```
const preco = 5000;
const quantidade = 2;

// Multiplicando
const total = preco * quantidade;
console.log(total);
```

Operadores: Multiplicação é feita com o asterisco (*).

EXECUÇÃO E SAÍDA

Resultado no Console

Ao executar o código, o interpretador resolve e imprime a saída:

SAÍDA ESPERADA:

```
> 10000
```

✓ **Dinâmica de Tipos:** Resultado inteiro é inferido como Number.

REFERÊNCIA RÁPIDA

Operadores JS

- `+` Adição
- `-` Subtração
- `*` Multiplicação
- `/` Divisão
- `%` Resto (Módulo)
- `**` Exponenciação
- `++` Incremento
- `--` Decremento



Estruturas Condicionais

Como tomar decisões no fluxo de execução utilizando if e else

CONCEITO BÁSICO

Tomando Decisões

O bloco `if` testa uma condição. Se for verdadeira, executa o primeiro bloco; caso contrário (`else`), executa o segundo.

```
const idade = 20;

if (idade >= 18) {
console.log("Maior de idade");
} else {
console.log("Menor de idade");
}
```

Dica: Condicionais controlam o fluxo de execução dinamicamente.



OPERADORES LÓGICOS e COMPARATIVOS

Utilizados para combinar ou inverter condições lógicas:

CONECTIVOS LÓGICOS

- `&&` (E / AND)
- `||` (OU / OR)
- `!` (NÃO / NOT)

RELACIONAIS

- `>` Maior `<` Menor
- `>=` Maior ou igual
- `<=` Menor ou igual

IGUALDADE

- `==` Igualdade
- `===` Igualdade Estrita

DIFERENÇA

- `!=` Diferente
- `!==` Diferente Estrito

✓ **Dica:** Combine operadores lógicos para criar condições de decisão complexas.

Estruturas de Repetição

Como executar blocos de código repetidamente utilizando while e for

CONCEITO BÁSICO

Laço While

O bloco **`while`** testa uma condição antes de cada iteração. Enquanto ela for verdadeira, o código é repetido.

```
let contador = 0;

while (contador < 3) {
  console.log(contador);
  contador++;
}
```

Dica: Evite loops infinitos garantindo a atualização da condição.

CONTROLE DE FLUXO COM FOR

Ideal quando sabemos previamente o número de repetições:

SINTAXE DO FOR

```
for (init; cond; inc) {
  // instruções
}
```

Consiste em três etapas controladas em uma única linha.

EXEMPLO PRÁTICO

```
for (let i=0; i<3; i++) {
  console.log(i);
}
```

Saída no console:
`> 0, 1, 2`

✓ **Dica:** O loop for agrupa inicialização, teste e incremento perfeitamente.

Funções em JavaScript

Como organizar, agrupar e reutilizar blocos de código de forma estruturada

CONCEITO BÁSICO

Declaração

Defina blocos de código reutilizáveis usando a palavra-chave **`function`**.

```
function saudacao(nome) {
  console.log("Olá, " + nome);
}
```

✓ **Reuso de Código:** Evita duplicações no fluxo do programa.

RETORNO DE VALOR

Retornando Valores

Use **`return`** para enviar um valor processado de volta para quem chamou a função.

```
function soma(a, b) {
  return a + b;
}
```

✓ **Saída de Dados:** Permite que o resultado seja guardado em variáveis.

DE FORMA PRÁTICA

Chamada e Resultado

Invoque a função enviando argumentos para obter o resultado correspondente:

```
const res = soma(5, 3);
```

```
console.log(res);
> 8
```

✓ **Organização:** Funções com retorno mantêm a lógica limpa e testável.



Porque Funções?

Comparação prática entre a repetição de código e o poder de reutilização das funções

PROBLEMA

Sem Função

Repetir as mesmas ações gera código redundante e dificulta manutenções futuras.

```
console.log("Olá, Ana");
console.log("Olá, João");
console.log("Olá, Maria");
```

Inconveniente: Cada nova saudação exige uma nova linha idêntica de código.

SOLUÇÃO

Com Função

Agrupe a lógica comum em um único bloco parametrizado e fácil de gerenciar.

```
function saudacao(nome) {
  console.log("Olá, " + nome);
}
```

Vantagem: A lógica do "Olá" é isolada e definida apenas uma vez na memória.

REUTILIZAÇÃO

Podemos Reutilizar

Chame a função passando diferentes argumentos para executar o mesmo comportamento.

```
saudacao("Ana");
saudacao("João");
saudacao("Maria");
```

Resultado: Código muito mais limpo, expressivo, dinâmico e fácil de manter.

O que é o DOM?

Como o navegador estrutura e manipula documentos HTML de forma dinâmica

CONCEITO BÁSICO

Document Object Model

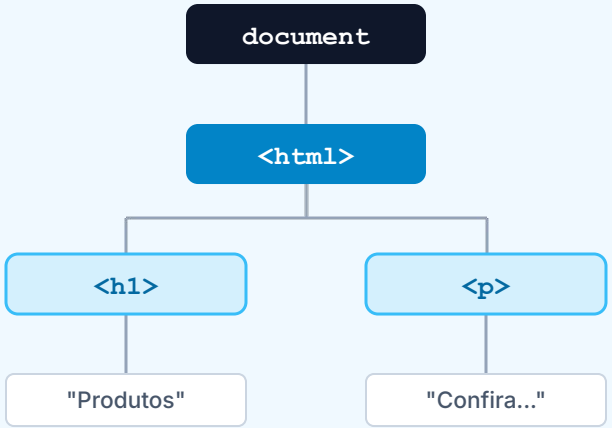
O navegador transforma o HTML em uma estrutura de objetos que pode ser manipulada e alterada dinamicamente pelo JavaScript.

```
<!-- HTML Original -->
<h1>Produtos</h1>
<p>Confira nossos produtos.</p>
```

Estrutura: O DOM funciona como uma ponte entre o HTML e o código JavaScript.

ESTRUTURA EM ÁRVORE

A Árvore do DOM



✓ **Visualização:** Cada tag HTML e texto vira um nó na árvore de objetos.

HTML → DOM

O fluxo de tradução e o papel do JavaScript na manipulação da página

PASSO A PASSO

Fluxo de Execução

1. HTML
O arquivo com as marcações estruturais da página.

2. Navegador interpreta
Lê o código e monta a estrutura em memória.

3. DOM (Document Object Model)
A representação viva e estruturada da página.

4. JavaScript manipula
Acessa e altera os elementos dinamicamente.

CONCEITO FUNDAMENTAL

Definição de DOM

O DOM é a representação interna da página que o JavaScript consegue acessar e modificar em tempo de execução.

```
// Exemplo de manipulação do DOM
const titulo = document.querySelector('h1');
titulo.textContent = "DOM Manipulado!";
```

✓ **Dinamismo:** Altere textos, cores e elementos sem recarregar a página.



Encontrando um elemento

Como selecionar e acessar elementos específicos do HTML utilizando o JavaScript

PASSO 1: ESTRUTURA

O Elemento no HTML

Para que um elemento possa ser selecionado facilmente, adicionamos um identificador único através do atributo `id`.

```
<!-- HTML Original -->
<h1 id="titulo">Produtos</h1>
```

✓ **Identificação:** O atributo ID deve ser exclusivo para cada elemento da página.

PASSO 2: SELEÇÃO

Acesso via JavaScript

Utilizamos o método `getElementById` para buscar e capturar a referência exata do elemento DOM.

```
// JavaScript
const titulo =
document.getElementById("titulo");
```

✓ **Vinculação:** A variável agora representa e permite manipular aquele elemento do DOM.

querySelector

A seleção de elementos de forma idêntica aos seletores que aprendemos no CSS

UMA FORMA MUITO UTILIZADA

Por ID (#)

Seleciona o primeiro elemento HTML que possui o atributo ID correspondente.

```
const titulo =
document.querySelector("#titulo");
```

Como no CSS: O caractere # indica que buscamos por um ID único.

TAMBÉM PODEMOS

Por Classe (.)

Seleciona o primeiro elemento que contém a classe especificada em sua estrutura.

```
const botao =
document.querySelector(".botao");
```

Como no CSS: O ponto inicial indica que buscamos por uma classe específica.

OU AINDA

Por Tag (Elemento)

Seleciona a primeira ocorrência da tag HTML informada na varredura do documento.

```
const paragrafo =
document.querySelector("p");
```

Como no CSS: Passar apenas o nome da tag seleciona o elemento diretamente.

CSS × querySelector

Os seletores CSS também podem ser utilizados para localizar elementos no DOM.

EXEMPLO 1: SELEÇÃO SIMPLES

Buscando por ID (#)

O método querySelector utiliza a mesma sintaxe de ID usada no CSS para identificar um elemento único.

```
/* CSS */
#titulo {
  color: blue;
}

// JavaScript
const titulo = document.querySelector("#titulo");
```

✓ **Unicidade:** O caractere # indica que buscamos por um identificador exclusivo.

EXEMPLO 2: SELEÇÃO COMPOSTA

Seletor Descendente

Permite selecionar elementos aninhados na estrutura HTML, seguindo a hierarquia de tags e classes.

```
/* CSS */
.card p { /* estilos */ }

// JavaScript
const p = document.querySelector(".card p");
```

✓ **Flexibilidade:** Permite aplicar qualquer regra de seleção do CSS diretamente no JS.

querySelectorAll()

Se quiser selecionar todos os elementos correspondentes, use querySelectorAll() para obter uma NodeList.

EXEMPLO PRÁTICO



Manipulação em Lote

Itere sobre a lista retornada para modificar as propriedades de todos os elementos.

```
<!-- HTML -->
<p>Parágrafo 1</p>
<p>Parágrafo 2</p>
<p>Parágrafo 3</p>

// JavaScript
const paragrafos =
  document.querySelectorAll("p");
paragrafos.forEach(p => p.style.color = "blue");
```

✓ **Resultado:** Todos os três parágrafos selecionados ficam azuis.

DIFERENÇA E FLEXIBILIDADE

Retorno Múltiplo vs CSS

Diferente de querySelector, ele retorna todos os elementos e aceita qualquer seletor CSS.

```
// querySelector vs querySelectorAll
querySelector("p")      → Primeiro <p>
querySelectorAll("p")   → Todos os <p>

document.querySelectorAll(".produto"); //class
document.querySelectorAll("#lista p"); //id e tag
```

✓ **Seletores:** Aceita classes, IDs, tags e seletores compostos do CSS.

Manipulando conteúdo

Utilizando a propriedade `textContent` para ler ou alterar o texto de um elemento do DOM

IMPLEMENTAÇÃO

Código HTML & JS

```
<!-- HTML Original -->
<h1 id="titulo">Produtos</h1>
```

```
// JavaScript
const titulo = document.querySelector("#titulo");
titulo.textContent = "Meus produtos";
```

✓ **Propriedade:** textContent define ou retorna o conteúdo de texto do nó.

EXECUÇÃO

Antes e Depois

ESTADO INICIAL (Antes)

Produtos

↓

ESTADO FINAL (Depois)

Meus produtos

✓ **Dinamismo:** O conteúdo é atualizado instantaneamente no navegador.

Alterando HTML

Como atualizar o conteúdo de um elemento interpretando as tags HTML

IMPLEMENTAÇÃO

Uso do innerHTML



```
// JavaScript
const mensagem =
   document.querySelector("#mensagem");
mensagem.innerHTML =
  "<strong>Compra   realizada!</strong>";
```

✓ **innerHTML:** Interpreta e renderiza as tags HTML contidas na string de forma dinâmica.

COMPARAÇÃO

`textContent` vs `innerHTML`

COM textContent (Trata como texto comum)

`<strong>Compra realizada!</strong>`

COM innerHTML (Interpreta as tags HTML)

Compra realizada!

✓ **Resultado:** O navegador processa a tag deixando o texto final em negrito.

Alterando atributos

Como ler e modificar os atributos de elementos HTML dinamicamente via JavaScript

IMPLEMENTAÇÃO

Código HTML & JS



```
<!-- HTML -->
<img id="foto" src="produto1.jpg">

// JavaScript
const foto = document.querySelector("#foto");
foto.src = "produto2.jpg";
foto.alt = "Notebook";
```

✓ **Atributos:** Qualquer atributo (src, alt, href, etc.) pode ser acessado e alterado diretamente.

EXECUÇÃO

Antes e Depois

ESTADO INICIAL (Antes)

`src="produto1.jpg" alt=""`

ESTADO FINAL (Depois)

`src="produto2.jpg" alt="Notebook"`

✓ **Modificação:** O navegador atualiza os atributos e renderiza o novo estado instantaneamente.

O método `setAttribute`

Uma forma alternativa e robusta para definir valores de atributos no HTML

IMPLEMENTAÇÃO

Sintaxe formal

```
// JavaScript
const foto = document.querySelector("#foto");
foto.setAttribute("src", "produto2.jpg");
foto.setAttribute("alt", "Notebook");
```

✓ **Atributos:** Permite manipular qualquer atributo HTML padrão ou customizado.

COMPARAÇÃO

Atribuição Direta vs setAttribute

PROPRIEDADE DIRETA (Mais simples)

```
foto.src = "produto2.jpg";
```

MÉTODO setAttribute (Mais formal)

```
foto.setAttribute("src", "p2.jpg");
```

✓ **Equivalência:** Ambos alteram o mesmo elemento DOM de forma imediata e visual.

Manipulando CSS - Alterando estilos

Como ler e modificar as propriedades de estilo (CSS) de elementos HTML diretamente via JavaScript

IMPLEMENTAÇÃO

Código HTML & JS

```
<!-- HTML -->
<p id="mensagem">Produto disponível</p>

// JavaScript
const mensagem = document.querySelector("#mensagem");
mensagem.style.color = "green";
```

✓ **Propriedade style:** Permite acessar e modificar diretamente qualquer propriedade de estilo CSS do elemento.

EXECUÇÃO

Antes e Depois

ESTADO INICIAL (Antes)

Produto disponível

ESTADO FINAL (Depois - Estilizado)

Produto disponível

✓ **Dinamismo:** O JavaScript altera diretamente o estilo CSS inline, que é atualizado na hora pelo navegador.

Várias propriedades

Alterar classes CSS em vez de construir toda a aparência diretamente com JavaScript

ABORDAGEM DIRETA

Estilos Inline com .style

```
// JavaScript direto
mensagem.style.color = "green";
mensagem.style.fontSize = "20px";
mensagem.style.fontWeight = "bold";
```

× **Desvantagem:** Mistura a lógica de comportamento com estilo e dificulta a manutenção do design.

RECOMENDADO

Uso de Classes CSS

```
/* No CSS */
.sucesso { color: green; font-size: 20px; ... }
// No JavaScript
mensagem.classList.add("sucesso");
```

✓ **Vantagem:** Código JS muito mais limpo e total separação entre estilo (CSS) e lógica (JS).

O objeto `classList`

Como adicionar e gerenciar classes CSS em elementos HTML via JavaScript dinamicamente

IMPLEMENTAÇÃO

Código HTML, CSS & JS



```
<!-- HTML -->
<p id="mensagem">Produto disponível</p>

/* CSS */
.sucesso { color: green; font-weight: bold; }

// JavaScript
mensagem.classList.add("sucesso");
```

✓ **Método classList:** Permite manipular as classes do elemento de forma limpa, sem reescrever estilos inline.

EXECUÇÃO

Antes e Depois

ESTADO INICIAL (Antes)

Produto disponível

ESTADO FINAL (Depois - Com Classe)

Produto disponível

✓ **Modificação:** O elemento **recebe a classe "sucesso"** e assume a estilização dinâmica do arquivo CSS.

Manipulando classes

Gerenciamento dinâmico de classes CSS em elementos HTML via classList



MÉTODO .add()

Adicionar

Adiciona uma ou mais classes ao elemento de forma simples.

```
element.classList.add("atv");
```

MÉTODO .toggle()

Alternar

Adiciona a classe se ela não existir, ou a remove se já existir.

```
element.classList.toggle("atv");
```

MÉTODO .remove()

Remover

Remove uma ou mais classes especificadas do elemento.

```
element.classList.remove("atv");
```

MÉTODO .contains()

Verificar

Verifica se a classe existe, retornando true ou false.

```
element.classList.contains("atv");
```

O que é um evento?

Um evento representa algo que acontece no navegador.

Clique
Interações de clique do usuário em elementos.

Movimento do mouse
Detecção de movimento ou arrastar do cursor.

Tecla pressionada
Entradas e interações através do teclado.

Formulário enviado
Envio ou submissão de dados de formulários.

Campo alterado
Mudança de valores ou foco em inputs HTML.

Página carregada
Finalização do carregamento do documento.

⚡ O JavaScript pode **reagir a esses acontecimentos** para tornar as páginas interativas.

Primeiro evento

Como escutar e reagir a interações do usuário utilizando `addEventListener`

HTML

O Elemento

```html
<!-- Botão alvo -->
<button id="botao">
  Clique aqui
</button>
```

JAVASCRIPT

O Escutador

```javascript
const botao =
    document.querySelector("#botao");

// 1. Função Anônima
botao.addEventListener(
  "click",
  function(){
    console.log("Clicou!");
  }
);

// 2. Função Nomeada
function aoClicar(){
  console.log("Clicou!");
}
botao.addEventListener("click",aoClicar);
```

FLUXO

Execução



1. Clique
Usuário clica no botão

2. Evento "click"
O navegador dispara o evento

3. Função Executada
O callback associado roda

4. "Clicou!"
Mensagem impressa no console

Evento `click`

Como responder a interações e alterar dinamicamente o conteúdo da página

IMPLEMENTAÇÃO

Código JavaScript



```
const titulo = document.querySelector("#titulo");
const botao = document.querySelector("#botao");

botao.addEventListener(
  "click",
  function () {
    titulo.textContent = "Você clicou!";
  }
);
```

EXECUÇÃO

Antes e Depois

ESTADO INICIAL (Antes)

Produtos

↓

ESTADO FINAL (Depois - Ao Clicar)

Você clicou!

✓ **Dinamismo:** A propriedade **textContent** altera o texto renderizado imediatamente.

Diretamente no HTML — `onclick`

Como acionar comportamentos JavaScript diretamente nos atributos de tags HTML

CONEXÃO DIRETA

HTML + JavaScript

```html
<!-- Botão com atributo de evento -->
<button onclick="aoClicar()">
  Clique aqui
</button>
```

↓

```javascript
// Função declarada no script
function aoClicar() {
  console.log("Clicou!");
}
```

BOAS PRÁTICAS

Separação de Responsabilidades

Estrutura vs Comportamento:

O HTML deve cuidar apenas da marcação estrutural, enquanto o JavaScript gerencia as interações.

⭐ **Preferência:** O uso de `addEventListener` é recomendado para manter o código limpo, modular e desacoplado.

⚠ ATENÇÃO AO REFERENCIAR FUNÇÕES:

`.addEventListener("click", aoClicar)`
➔ Correto: passa apenas a referência (será executada ao clicar)

`.addEventListener("click", aoClicar())`
➔ Incorreto: executa a função imediatamente ao carregar

Evento `mouseover`

Como detectar e reagir à passagem do ponteiro do mouse sobre os elementos

IMPLEMENTAÇÃO

Código JavaScript



```
card.addEventListener(
  "mouseover",
  function () {
    console.log("Mouse sobre o card");
  }
);
```

REFERÊNCIA

Outros Eventos de Mouse

- **`click`** — Disparado ao clicar no elemento
- **`dblclick`** — Clique duplo rápido
- **`mouseover`** — Cursor entra no elemento
- **`mouseout`** — Cursor deixa o elemento
- **`mousedown`** — Botão do mouse é pressionado
- **`mouseup`** — Botão do mouse é solto

Eventos de teclado

Como detectar e reagir às interações do usuário através das teclas no navegador

IMPLEMENTAÇÃO

Código JavaScript



```
input.addEventListener(
  "keydown",
  function () {
    console.log("Tecla pressionada");
  }
);
```

REFERÊNCIA

Eventos Comuns

keydown
Disparado no momento em que uma tecla é pressionada. ***Ideal para ações de resposta imediata ou atalhos de teclado.***

keyup
Disparado quando o usuário solta uma tecla que estava pressionada. ***Comumente utilizado para validação de campos de texto.***

JavaScript + formulário

Como capturar e controlar o envio de dados e interações do usuário

HTML

O Formulário



```html
<!-- Estrutura básica -->
<form id="formulario">
  <input id="nome">
  <button type="submit">
    Enviar
  </button>
</form>
```

JAVASCRIPT

O Evento de Envio

```javascript
const formulario =
document.querySelector("#formulario");

formulario.addEventListener(
  "submit",
  function (event) {
    // Controla o envio aqui
    ...
  }
);
```

Impedindo o comportamento padrão

Como evitar o recarregamento automático da página no envio de formulários

IMPLEMENTAÇÃO

Código JavaScript

```
formulario.addEventListener(
  "submit",
  function (event) {
    // Impede o envio padrão
    event.preventDefault();
  }
);
```

COMPORTAMENTO

Controle do Fluxo

Ação Padrão: Ao enviar um formulário, o navegador normalmente realiza o envio dos dados e recarrega a página.

O Bloqueio: Chamando `event.preventDefault()` nós dizemos ao navegador para interromper essa ação automática.

✓ **Liberdade Total:** Agora o JavaScript assume o controle completo para validar os campos, enviar via AJAX/Fetch e atualizar a UI sem recarregar.

Obtendo o valor de um campo

Como capturar e utilizar o conteúdo digitado pelo usuário em elementos de formulário

IMPLEMENTAÇÃO

Código Fonte

```
<!-- HTML -->
<input id="nome">

// JavaScript
const nome = document.querySelector("#nome");
console.log(nome.value);
```

COMPORTAMENTO

Fluxo dos Dados

Se o usuário digitou no campo:

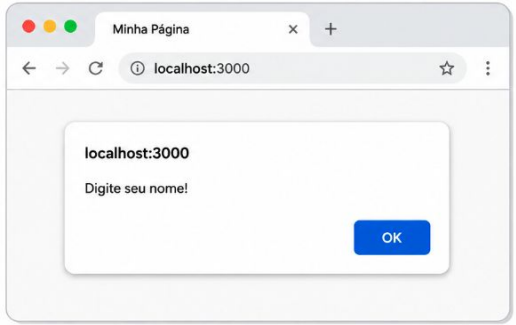
Lucas

Teremos o seguinte valor capturado:

`nome.value == "Lucas"`

Validação simples

Como verificar dados essenciais do usuário e impedir o envio caso estejam inválidos



IMPLEMENTAÇÃO

Código JavaScript



```
formulario.addEventListener(
  "submit",
  function (event) {
    if (nome.value === "") {
      event.preventDefault();
      alert("Digite seu nome!");
    }
  }
);
```

COMPORTAMENTO

Fluxo de Execução

1. Clique em Enviar:
O usuário tenta submeter o formulário clicando no botão de envio.

2. Teste da Condição:
O script verifica se o campo de texto está vazio (`nome.value === ""`).

3. Desvio de Ações:

- **NÃO preenchido:** Para o envio (`preventDefault`) e mostra alerta.
- **SIM, preenchido:** Ignora o bloco if e continua para o servidor.

Criando elementos dinamicamente

Como criar novos elementos HTML via JavaScript e definir suas propriedades na memória

IMPLEMENTAÇÃO

Código JavaScript

```
// JavaScript também pode criar HTML
const item = document.createElement("li");

item.textContent = "Notebook";
```

COMPORTAMENTO

Criação em Memória

1. Criação do Elemento:
O método `document.createElement("li")` cria o elemento `<li>` apenas na memória.

2. Definição de Conteúdo:
Com `item.textContent` definimos o texto interno do novo elemento dinamicamente.

⚠ Estado Pendente:
O elemento existe na memória do JS, mas **ainda não está na página**. Precisamos adicioná-lo ao DOM.

Adicionando elementos à página

Como inserir elementos criados dinamicamente na estrutura do DOM usando o método `append`

IMPLEMENTAÇÃO

Código JavaScript



```
<!-- HTML -->
<ul id="produtos"></ul>

// JavaScript
const lista = document.querySelector("#produtos");
const item = document.createElement("li");
item.textContent = "Notebook";
lista.append(item);
```

COMPORTAMENTO

Fluxo dos Dados

1. Seleção do Container:
O JS busca o elemento pai existente na página através do seletor `#produtos`.

2. Associação (append):
O método `append()` adiciona o novo item como o último elemento filho da lista.

Resultado na Página:

- `Notebook`

Criando vários elementos

Como automatizar a criação e inserção de múltiplos elementos utilizando estruturas de repetição

IMPLEMENTAÇÃO

Código JavaScript

```
const produtos = ["Notebook", "Celular", "Tablet"];

produtos.forEach(function (prod) {
  const item = document.createElement("li");
  item.textContent = prod;
  lista.append(item);
});
```

COMPORTAMENTO

Renderização no DOM

1. Iteração em Lote:
O loop percorre individualmente cada item string definido no array original.

2. Geração de Filhos:
Múltiplos elementos `<li>` são instanciados e anexados consecutivamente à lista pai.

Resultado na Página:

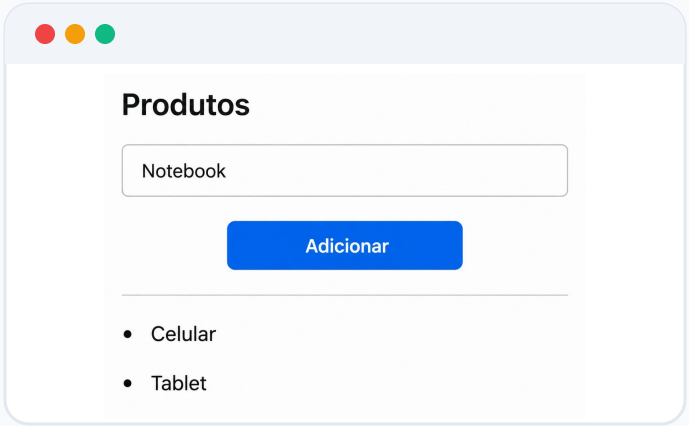
- Notebook
- Celular
- Tablet

Miniaplicação: Interação

Como o evento de clique dispara o fluxo de atualização dinâmica do DOM

CONCEITO VISUAL

O que vamos criar



COMPORTAMENTO

Fluxo ao clicar em Adicionar

1. **Usuário** clica no botão de adicionar.
2. **Evento click** é disparado pelo navegador.
3. **JavaScript** detecta a ação via Event Listener.
4. **Cria ** novos elementos dinamicamente na memória.
5. **DOM** recebe a inserção do novo elemento filho.
6. **Interface Atualizada** com o novo item renderizado na tela.

Miniaplicação: HTML

A estrutura de marcação que serve como base para a nossa interface interativa

ESTRUTURA DE TELA

Código HTML

```html
<!-- Campo de entrada -->
<input id="produto">

<!-- Botão para acionar -->
<button id="adicionar">Adicionar</button>

<!-- Lista de itens vazia -->
<ul id="lista"></ul>
```

COMPONENTES

Elementos de Interface

1. Campo de Entrada:
O elemento `#produto` é o campo de texto onde o usuário digitará os novos produtos para a lista.

2. Botão de Adicionar:
O botão `#adicionar` servirá de gatilho de clique para rodar a função JS de inserção.

3. Lista Dinâmica:
O container `#lista` inicia vazio no HTML e receberá os novos itens filhos criados via JS.

Miniaplicação: JavaScript

O código completo para integrar elementos HTML, eventos e manipulação dinâmica do DOM

IMPLEMENTAÇÃO

Código JavaScript



```
const input = document.querySelector("#produto");
const botao = document.querySelector("#adicionar");
const lista = document.querySelector("#lista");

botao.addEventListener("click", function () {
  const item = document.createElement("li");
  item.textContent = input.value;
  lista.append(item);
  input.value = "";
});
```

COMPORTAMENTO

Funcionamento do Script

1. Captura de Elementos:
O JS obtém referências para os elementos do HTML usando o método `querySelector`.

2. Escuta de Eventos:
O método `addEventListener` monitora cliques no botão para executar a função de inserção.

3. Manipulação do DOM:
A cada clique, um novo `<li>` é gerado com o texto do input, anexado à lista e o campo de texto é limpo.

O que aconteceu?

A análise passo a passo de como a interação foi processada e os conceitos integrados

MECANISMO

Fluxo de Execução

1. Clique: O usuário clica no botão "Adicionar".

2. Event Listener: O navegador dispara a escuta do clique.

3. Lê *input.value*: O script lê o texto inserido no campo.

4. Cria <li>: Um novo elemento de lista é criado em memória.

5. Define textContent: O texto capturado é atribuído ao item.

6. append(): O novo item é inserido dentro da lista pai.

7. DOM Atualizado: A interface exibe o novo elemento de forma dinâmica.

TECNOLOGIAS

Conceitos Integrados

Este pequeno exemplo reúne as bases fundamentais do desenvolvimento Front-End:

- **JavaScript:** Lógica, variáveis e manipulação de fluxos.
- **DOM:** A estrutura de objetos que representa a interface HTML.
- **Eventos:** Mecanismo para capturar e reagir à ação do usuário.
- **Formulários:** Entrada e controle de dados digitados em tela.
- **Manipulação:** Criação de novos elementos e atualização da tela.

JS não precisa desenhar a interface

JavaScript controla o estado | CSS controla a apresentação.

JAVASCRIPT

Decide o que acontece

```
// Altera o estado do elemento

mensagem.classList.add("erro");
```

CSS

Define como aparece

```
/* Aplica a estilização */

.erro {
  color: red;
  font-weight: bold;
}
```

Além do JavaScript: Interfaces Ricas

Tecnologias para aplicações complexas e interativas

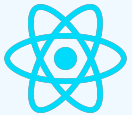
Quando o JavaScript puro (**Vanilla JS**) torna-se complexo para gerenciar grandes estados de aplicação, utilizamos Frameworks e Bibliotecas para criar interfaces altamente dinâmicas.

Foco da Disciplina

Estas tecnologias **não serão abordadas** nesta disciplina. O foco aqui é o domínio dos fundamentos do **JavaScript puro**, que é a base indispensável para compreender como qualquer uma dessas ferramentas funciona por baixo dos panos.

Ecossistema Moderno

React (Meta)
Foco em componentes reutilizáveis e fluxo de dados unidirecional.



Vue.js (Comunitário)
Foco na simplicidade e facilidade de integração incremental.



Angular (Google)
Plataforma robusta para aplicações corporativas escaláveis.



Comunicação com o Servidor

A evolução do processamento local do JavaScript para a arquitetura cliente-servidor

Modelo Local

Até Agora

1. Usuário: Realiza interações diretamente na página

2. Navegador: Captura e processa os eventos gerados

3. JavaScript: Executa toda a lógica localmente

4. DOM: Atualiza a interface diretamente na tela

Modelo Cliente-Servidor

Comunicação Remota

1. Navegador: Inicia requisições assíncronas

2. Protocolo HTTP: Transporta os dados pela rede

3. Servidor: Processa as requisições no back-end

4. API REST: Define as rotas e regras de integração

Importante: Esta transição de fluxo será fundamental quando entrarmos no desenvolvimento de **back-end e Phoenix**.

fetch

Uma forma moderna de realizar requisições HTTP

SINTAXE

Exemplo de Código

```
// Realiza a requisição HTTP
fetch("http://localhost:3000/api/products")
  .then(res => res.json())
  .then(products => {
    console.log(products);
  });
```

CONCEITUALMENTE

Fluxo de Comunicação

1. **JavaScript:** Inicia requisição assíncrona
2. **GET /api/products:** Envia a rota para a rede
3. **Servidor:** Recebe e processa a requisição
4. **JSON:** Servidor retorna os dados formatados
5. **JavaScript:** Consome os dados e atualiza a tela

Aqui começamos a conectar o **front-end** ao **back-end** .

Detalhes do exemplo anterior

Entendendo o fluxo de execução do `fetch()` passo a passo

SINTAXE

Código Analisado

```
fetch("http://localhost:3000/api/products")
  .then(res => res.json())
  .then(products => {
    console.log(products);
});
```

```
products = [
    { id: 1, name: "Notebook" },
    { id: 2, name: "Celular" },
    { id: 3, name: "Tablet" }
]
```

1 **fetch("/api/products")**

Faz uma requisição HTTP **GET** padrão para buscar os dados de produtos no servidor.

2 **.then(res ⇒ res.json())**

Como a resposta do servidor chega em JSON assincronamente (Promise), converte-a em objeto JS.

3 **.then(products ⇒ { ... })**

Recebe os produtos convertidos e executa o **console.log()** para fins de depuração.

Detalhes do exemplo anterior

Entendendo o fluxo de execução do `fetch()` passo a passo

SINTAXE

Código Analisado

```
fetch("http://localhost:3000/api/products")
  .then(res => res.json())
  .then(products => {
    console.log(products);
    console.log(products[0].name);
});
```

Notebook

1 **fetch("/api/products")**
Faz uma requisição HTTP **GET** padrão para buscar os dados de produtos no servidor.

2 **.then(res ⇒ res.json())**
Como a resposta do servidor chega em JSON assincronamente (Promise), converte-a em objeto JS.

3 **.then(products ⇒ { ... })**
Recebe os produtos convertidos e executa o **console.log()** para fins de depuração.

Atualização do Front-End

Buscando dados no Back-End e atualizando o DOM dinamicamente

```
// 1. Busca os dados da API
function clientServerExample() {
  fetch("http://localhost:3000/api/products")
    .then(res => res.json())
    .then(products => {
      console.log(products);

      // Renderiza no HTML
      renderJSON(products);
    });
}
```

```
// 2. Manipula o DOM para exibir
function renderJSON(products) {
  const lista = document.querySelector("#lista-produtos");

  products.forEach(function(p) {
    const item = document.createElement("li");
    item.textContent = p.name;
    lista.append(item);
  });
}
```

Clique aqui!

```
<button onclick="clientServerExample()">Clique aqui!</button>
```

Exemplo Back-end

Construindo uma API simples em Node.js com o módulo nativo HTTP

```
const http = require("http");
const products = [
  { id: 1, name: "Notebook" },
  { id: 2, name: "Celular" },
  { id: 3, name: "Tablet" }
];
const server = http.createServer((req, res) => {
  if (req.url === "/api/products" && req.method === "GET") {
    res.writeHead(200, {"Content-Type": "application/json",
                        "Access-Control-Allow-Origin": "*"} );
    res.end(JSON.stringify(products));
  } else {
    res.writeHead(404, { "Content-Type": "text/plain" });
    res.end("Rota não encontrada");
  }
});
server.listen(3000, () => {
  console.log("Servidor rodando...");
});
```

1

Criação do Servidor

O método `createServer` do módulo HTTP inicializa o servidor Node.js que escutará as requisições na porta 3000.

Inicia Back-end

```
node server.js
```

2

Roteamento e Resposta

Filtra requisições **GET /api/products** para retornar a lista de produtos em formato JSON (status 200). Qualquer outra rota resulta em erro 404.



JavaScript no DevTools

Como abrir e testar comandos no console do próprio navegador

PASSO A PASSO

Como Acessar

1. Atalho: Pressione **F12** (ou Fn+F12) para abrir as ferramentas de desenvolvedor.

2. Seleção: Clique na aba **Console** no painel superior do DevTools.

3. Interação: Comece a digitar comandos JS e pressione Enter para executá-los.

EXPERIMENTAÇÃO

Comandos de Teste no DOM



```
document // Retorna o documento HTML inteiro

document.body // Acessa o nó do corpo

document.querySelector("h1") // Busca título
principal

document.querySelector("h1").textContent = "Novo
título";
```

Efeito Imediato: O DOM será alterado imediatamente na sua aba do navegador ao executar os comandos!

PARADIGMAS EM JS

JavaScript e Programação Funcional

Uma introdução prática aos conceitos de funções como cidadãs de primeira classe, imutabilidade e a transição do paradigma imperativo para o declarativo.

O que vamos ver:

1. Funções como valores
Tratar funções como dados (*First-Class Citizens*)

2. Imutabilidade de dados

3. Métodos Declarativos
Utilizar `map`, `filter` e `forEach` de forma prática

JavaScript e Programação Funcional

JavaScript permite diferentes estilos e paradigmas de programação.

Multiparadigma

Estilos de Programação

imperativo

orientado a objetos

FUNCIONAL
Foco exclusivo desta parte da aula

Nesta parte da Aula

Objetivos do Aprendizado

1. Elementos Práticos: Vamos conhecer apenas alguns elementos de programação funcional presentes em JavaScript.

2. Visão Conceitual: O objetivo não é estudar o paradigma funcional a fundo, mas reconhecer algumas de suas ideias principais.

Funções como valores

Em JavaScript, funções são cidadãs de primeira classe e podem ser tratadas como dados.

EXEMPLOS EM JAVASCRIPT

Armazenando Funções

```
// Expressão de função tradicional
const saudacao = function(nome) {
  return "Olá, " + nome;
};

console.log(saudacao("Lucas"));

// Equivalente com Arrow Function
const saudacaoArrow = (nome) => "Olá, " + nome;
```

PROPRIEDADES



Uma função pode ser:

1. **Armazenada:** em variáveis, arrays ou propriedades de objetos.
2. **Passada como argumento:** para outras funções *(callbacks)*.
3. **Retornada:** como resultado de execução de outra função.

CONCEITO CHAVE

Funções são valores que podem ser manipulados livremente pelo programa.

Funções como argumentos

Podemos passar uma função para outra função.

EXEMPLO PRÁTICO

Passando Callbacks em JS

```
// Declaração da função
function executar(funcao) {
  funcao();
}

// Passando função anônima
executar(function() {
  console.log("Olá!");
});
```

FLUXO MENTAL

Como o interpretador pensa:



1. chamar `executar()`: Dispara a execução da função principal.
2. receber a função: O parâmetro "funcao" aceita a função anônima.
3. executar a função: Invoca o callback internamente via `funcao()`.

CONCEITO CHAVE

Essencial para Métodos Funcionais

`map()`
`filter()`
`forEach()`

Estes métodos de array dependem diretamente do envio de callbacks.

forEach()

Uma função pode ser executada para cada elemento de uma coleção.

Exemplo de Código

Sintaxe JS tradicional vs Arrow

```
const produtos = ["Notebook", "Celular", "Tablet"];
// Execução tradicional
produtos.forEach(function(p) {
  console.log(p);
});
// Equivalente com Arrow Function
produtos.forEach(p => console.log(p));
```

Funcionamento

Como o método opera:

`forEach()` recebe uma **função callback** que é disparada individualmente para cada elemento iterado dentro do array.

Resultado no Console

```
> Notebook
> Celular
> Tablet
```

map()

Transforme cada elemento de uma coleção criando um novo array.

EXEMPLO PRÁTICO

Sintaxe e Uso do map()

```
const precos = [100, 200, 300];

// Cria um novo array transformado
const comDesconto = precos.map((p) => {
  return p * 0.9;
});
```

FLUXO DE TRANSFORMAÇÃO

Visualizando o Mapeamento



Original: `[100, 200, 300]`

↓ `map(preco => preco * 0.9)`

Novo: `[90, 180, 270]`

CONCEITO CHAVE

Imutabilidade & Retorno

Nova Coleção: O array original permanece intacto. Um novo array com os elementos modificados é gerado.

Mapeamento Unívoco: Cada item da entrada corresponde a exatamente um item na saída.

filter()

Selecione elementos de uma coleção que atendem a uma condição específica.

EXEMPLO PRÁTICO

Sintaxe e Uso do filter()

```
const precos = [100, 500, 200, 800];
// Filtra os valores maiores ou iguais a 500
const caros = precos.filter((p) => {
  return p >= 500;
});
```

FLUXO DE SELEÇÃO

Visualizando a Filtragem



Original: `[100, 500, 200, 800]`

↓ `filter(preco => preco >= 500)`

Filtrado: `[500, 800]`

CONCEITO CHAVE

Seleção Condicional & Nova Coleção

Subconjunto de Dados: Mantém apenas os elementos do array original que retornam verdadeiro para o critério estabelecido.

Imutabilidade: Retorna uma nova coleção sem realizar nenhum tipo de alteração no array de origem.

`map()` × `filter()`

Demonstração prática de seleção e transformação em coleções

DADOS DE ORIGEM

Array de Produtos

```
const produtos = [
  { nome: "Notebook", preco: 5000 },
  { nome: "Mouse", preco: 100 },
  { nome: "Teclado", preco: 300 }
];
```

FILTRAGEM

`produtos.filter()`

Filtra os elementos mantendo apenas os que atendem ao critério:

```
const caros = produtos.filter(
   prod => prod.preco > 500
);
```

RESULTADO:

```
[
 { nome: "Notebook", preco: 5000 }
]
```

TRANSFORMAÇÃO

`produtos.map()`

Mapeia os elementos originais criando uma lista com os novos valores:

```
const nomes = produtos.map(
   prod => prod.nome
);
```

RESULTADO:

```
["Notebook", "Mouse", "Teclado"]
```

Estilo Funcional

Como combinar e encadear operações sobre coleções de dados

IMPLEMENTAÇÃO EM JAVASCRIPT

```
const produtos = [
  { nome: "Notebook", preco: 5000 },
  { nome: "Mouse", preco: 100 },
  { nome: "Teclado", preco: 300 },
  { nome: "Monitor", preco: 1200 }
];

const nomesCaros = produtos
  .filter(p => p.preco > 500)
  .map(p => p.nome);
```

RESULTADO (nomesCaros):

```
[ "Notebook", "Monitor" ]
```

FLUXO DE TRANSFORMAÇÃO

Pipeline de Execução



1. **Entrada:** `produtos` (Lista original com 4 itens)

↓ `.filter(p => p.preco > 500)`

2. **Filtragem:** Retorna Notebook e Monitor

↓ `.map(p => p.nome)`

3. **Projeção:** Extrai apenas a propriedade "nome"

⚡ **Conceito Chave:** Podemos construir transformações complexas e legíveis apenas encadeando funções!

Ponte para Elixir

Como a experiência com JavaScript nos prepara para o paradigma funcional

JAVASCRIPT FUNCIONAL

O que já vimos até aqui

Noções e métodos que você experimentou no console:

- **Funções como valores e argumentos**
- **Iteradores e Projeções:** `forEach()`, `map()`, `filter()`

Essas ideias são a base de qualquer linguagem funcional!

PRÓXIMA UNIDADE

Aprofundamento em Elixir

Onde a Programação Funcional é o padrão do ecossistema:

- ✦ **Imutabilidade estrita de dados**
- ✦ **Funções puras e de ordem superior completas**
- ✦ **Composição robusta e Pattern Matching**
- ✦ **Recursão como controle de fluxo**

Em Elixir, estas ideias são a parte central da linguagem!

RESUMO: O que aprendemos?

Resumo dos principais conceitos do desenvolvimento web com JavaScript

1. Fundamentos

SINTAXE E LÓGICA

- Variáveis `let/const`
- Condicionais `if/else`
- Estruturas de repetição
- Funções estruturadas

2. DOM

MANIPULAÇÃO DE PÁGINAS

- Selecionar elementos
- Modificar conteúdo
- Criar novos nós

3. Eventos

INTERATIVIDADE

- Clicks de botões
- Eventos de teclado
- Envios de formulário
- Ações do mouse

4. Integração HTTP

APIS E ASSINCRONISMO

- Requisições assíncronas
- Consumo de APIs
- `fetch()` e Promises

RESUMO: A evolução do Front-end

Como a estrutura se conecta ao comportamento e à comunicação com o servidor

01
HTML
Estrutura

O esqueleto da página. Define os elementos, conteúdo e a marcação sem estilização.

02
CSS
Aparência

O design visual. Responsável por cores, fontes, espaçamentos e layouts modernos.

03
JavaScript
Comportamento

A interatividade. Controla ações do usuário, manipulação dinâmica do DOM e eventos.

04
HTTP / API
Integração

A conexão externa. Permite que a aplicação web se comunique e salve dados no Back-end.

Front-end completo

Ao final desta unidade, os alunos devem conseguir enxergar uma aplicação Web assim:



Próxima etapa

Como construir o back-end dessa aplicação?

Tecnologias sugeridas

Elixir + Phoenix

Aprenda a criar APIs robustas, escaláveis e de alta performance para conectar ao seu Front-end completo.

Atividade prática

Tornando nossa página interativa |> Utilize a página de produtos criada anteriormente e implemente.

1. Botão "Comprar"

Funcionamento

Ao clicar no botão:

Comprar

↓

"Produto adicionado ao carrinho!"

2. Campo de busca

Interface & Filtro

[Digite um produto...]

Buscar

Ao clicar em **Buscar**, altere a lista de produtos apresentada em tela.

3. Formulário

Validação Obrigatória

Campos a validar:

- Nome
- E-mail
- Mensagem

Não permita o envio se algum campo obrigatório estiver vazio.

4. Interação visual

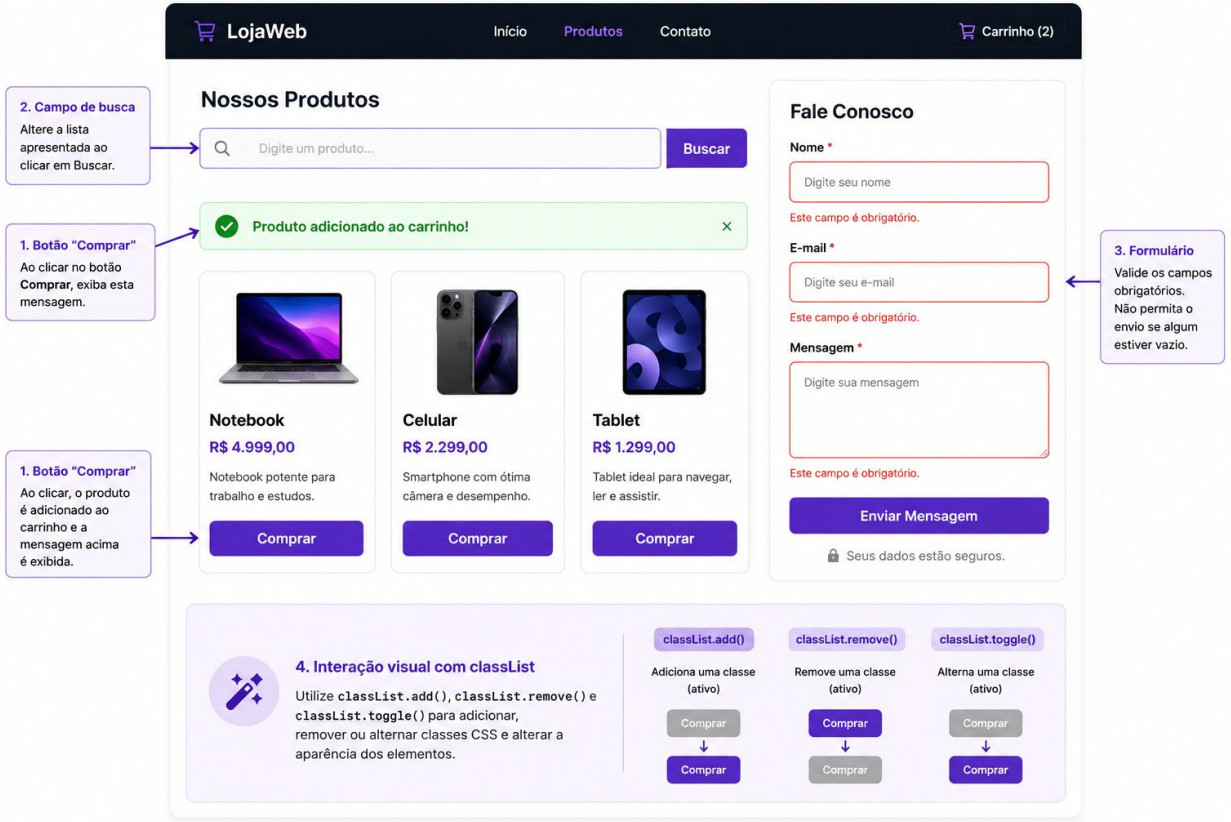
Manipulação do DOM

Utilize para alterar as classes CSS dos elementos:

```
classList.add()
classList.remove()
classList.toggle()
```

Importante: Não é necessário implementar nenhum back-end!

Exemplo: Atividade prática





Bons estudos!

Prof. Lucas Vegi